

Roteiro do Vídeo de Apresentação

Pokédex Interativa — Catálogo Interativo com a PokéAPI

Projeto Somativo de Flutter • Desenvolvimento Mobile Híbrido • Bacharelado em Sistemas de Informação

Grupo: Rogerio de Abreu Mar • Logan Borges • Felipe da Veiga Gomes • Alex Narok Stavasz

Como usar este roteiro. Cada bloco traz o intervalo de tempo, quem fala, o que precisa estar aparecendo na tela e os pontos que devem ser ditos. As falas em itálico são sugestões — adapte para o seu jeito de falar, mas mantenha os termos técnicos e os nomes de arquivo, porque é isso que comprova o domínio do código.

Todos os arquivos citados existem no repositório (commit 3bbd0f1) e os números mostrados na demonstração (catálogo indo até o #040, Favoritos com 3 e Capturados com 2) são os do app rodando de verdade no emulador.

1. Regras do vídeo (item 7 do enunciado)

Item	Regra
Hospedagem	YouTube, marcado como Público (não serve “não listado”).
Duração	O item 7.2 pede de 6 a 9 minutos, mas o item 8 desconta -5% por minuto abaixo de 6:00 ou acima de 8:00 . Por isso este roteiro fecha em 8:00 e a margem segura é de 7:00 a 8:00.
Participação	Todos os integrantes precisam aparecer e falar , e cada um deve explicar ao menos uma parte que implementou.
Abertura	Antes de qualquer outra coisa, cada integrante diz o nome completo , igual ao que está na capa do PDF de entrega. Nome que faltar na capa = zero para aquela pessoa.

2. Divisão do tempo entre os 4 integrantes

A divisão é contínua: cada pessoa assume um trecho inteiro do vídeo, do começo ao fim do seu bloco, em vez de revezar a cada assunto. Os cortes respeitam exatamente as faixas obrigatórias do item 7.3.

Tempo	Quem fala	Dura	Bloco
0:00 – 0:30	Todos	0:30	Apresentação: nome completo de cada integrante
0:30 – 2:00	Rogerio de Abreu Mar	1:30	Demonstração do app rodando
2:00 – 3:45	Logan Borges	1:45	Código: PokéAPI, catálogo, paginação e feedback de UI (RF01, RF02, RF03, RF09)
3:45 – 5:30	Felipe da Veiga Gomes	1:45	Código: Provider, persistência, login e busca (RF04 a RF08)
5:30 – 8:00	Alex Narok Stavasz	2:30	Explicação técnica em profundidade + encerramento

Atenção à regra do item 7.2. Como Rogerio e Alex não pegam o bloco de “explicação de código”, eles precisam citar explicitamente um arquivo que implementaram dentro do próprio bloco — o roteiro já indica onde (Rogerio em 0:38, e o bloco inteiro do Alex, que é sobre o código dele).

3. Roteiro detalhado

0:00 – 0:30 — Abertura (todos na câmera)

Na tela: os quatro integrantes na câmera. Nada de compartilhamento de tela ainda.

“Olá, professor. Este é o nosso projeto somativo de Flutter.”

Em seguida, um de cada vez, sem pressa (cerca de 5 segundos por pessoa). Diga o nome completo, exatamente como está na capa do PDF de entrega:

“Meu nome completo é Rogerio de Abreu Mar.”

“Meu nome completo é Logan Borges.”

“Meu nome completo é Felipe da Veiga Gomes.”

“Meu nome completo é Alex Narok Stavas.”

Fechando o bloco (quem estiver conduzindo a abertura):

“Nosso tema é Pokémon. O app se chama Pokédex Interativa e consome a PokéAPI.”

0:30 – 2:00 — Rogerio de Abreu Mar: demonstração do app

Na tela: gravação do emulador Android com o app já aberto na tela de login (saia da conta antes de começar a gravar). O ritmo aqui é apertado: ensaie até caber em 1:30.

Tempo	O que fazer na tela	O que dizer
0:30	Tela de login visível; digitar e-mail e senha e entrar.	O app exige login antes de liberar o catálogo, que é o RF07. Fizemos autenticação real com Supabase Auth, que é o bônus do enunciado.
0:38	Enquanto a grade carrega.	Quem decide se aparece o login ou o catálogo é o AuthGate, em <code>lib/app.dart</code> , que fui eu quem implementou.
0:45	Catálogo carregado, rolar um pouco a grade.	Este é o RF01: GridView com imagem e nome, vindo da PokéAPI.
1:00	Rolar até o fim e tocar em Carregar mais .	O botão busca a página seguinte e acrescenta à lista: saímos do #020 e fomos até o #040, sem recarregar o que já estava na tela.
1:12	Tocar em um card (ex.: Charizard) e rolar o detalhe até os status base.	Aqui é o RF02 e o RF03: navegação com Navigator e uma segunda requisição que traz descrição, tipos, altura, peso e status.
1:20	Tocar na estrela e depois em Capturar .	Favoritar é o RF04 e capturar é a lista de itens consumidos do RF07.
1:28	Voltar e abrir Favoritos e depois Capturados .	As duas listas são independentes e se atualizam sozinhas, porque leem o mesmo estado global.
1:40	Buscar pikachu; voltar e buscar xyz123.	A busca do RF08 vai direto para a tela de detalhes. Quando o nome não existe, aparece uma mensagem amigável, diferente da mensagem de falha de rede.
1:52	Fechar o app pelos recentes do Android e abrir de novo.	Fechando e abrindo, a sessão e as duas listas continuam lá: é o RF06, a persistência.

2:00 – 3:45 — Logan Borges: PokéAPI, catálogo e paginação

Na tela: compartilhe o editor com os arquivos abertos, um de cada vez, na ordem abaixo (caminhos relativos a `lib/`). **RFs cobertos:** RF01, RF02, RF03 e RF09.

Tempo	Arquivo	O que mostrar e dizer
2:00	<code>services/pokeapi_service.dart</code>	Método <code>fetchPage</code> : a paginação é feita com <code>limit</code> e <code>offset</code> , 20 itens por vez. Todo o tratamento de erro fica concentrado em <code>_getJSON</code> : timeout de 15 segundos, 404 marcado como <code>isNotFound</code> e qualquer outra falha virando uma mensagem pronta para a tela. Assim <code>provider</code> e telas só lidam com <code>PokeApiException</code> .

Tempo	Arquivo	O que mostrar e dizer
2:25	services/ pokeapi_service.dart	Método fetchDetail: são duas requisições, /pokemon/{id} e /pokemon-species/{id}, que é a segunda requisição pedida no RF03. A de species é best-effort: se ela falhar, o detalhe aparece sem a descrição em vez de quebrar a tela.
2:45	core/utils/ pokemon_utils.dart	A listagem da PokéAPI devolve só nome e URL, sem id e sem imagem. O idFromUrl extrai o id da própria URL e o artworkUrl monta o endereço da imagem a partir dele — isso evita 20 requisições extras só para desenhar uma página da grade.
3:05	providers/ catalog_provider.dart	Mostrar loadMore: ele usa _items.addAll, ou seja, acrescenta à lista existente. Se a rede cair no meio, a lista já carregada é preservada e só o errorMessage é preenchido.
3:25	screens/ home_screen.dart widgets/ pokemon_image.dart	No _LoadMoreFooter, os três estados do RF09: spinner enquanto carrega, mensagem de erro com o botão para tentar de novo, ou o botão Carregar mais. E no PokemonImage, o errorBuilder que mostra o placeholder — é o que garante a exigência do RF01 de que item sem imagem não quebre a tela.

3:45 – 5:30 — Felipe da Veiga Gomes: Provider, persistência, login e busca

Na tela: mesma dinâmica, arquivos abertos na ordem (caminhos relativos a lib/). **RFs cobertos:** RF04, RF05, RF06, RF07 e RF08.

Tempo	Arquivo	O que mostrar e dizer
3:45	providers/ collection_provider.dart	Este é o coração do RF04 e do RF06. Mostrar o método _apply, que faz três coisas em ordem: atualiza a memória e chama notifyListeners, grava no shared_preferences e só então envia para o Supabase. É uma estratégia local-first: a tela responde na hora e a nuvem é o último passo.
4:10	services/ local_storage_service.dart	A coleção é salva em JSON na chave collection:<userId>, ou seja, separada por usuário. E mesmo no login local a senha nunca é gravada em texto puro: guardamos o hash SHA-256 com o e-mail como sal.
4:30	services/ auth_service.dart app.dart	O AuthService tem dois modos: se as credenciais do Supabase foram compiladas, usa Supabase Auth; se não, cai no cadastro local. O baseline obrigatório nunca depende da nuvem. E o AuthGate faz a navegação condicional do RF07: o catálogo só existe na árvore de widgets quando há sessão.
4:50	screens/ detail_screen.dart	Aqui está o FutureBuilder recomendado pelo RF09, com os três estados: carregando, erro com botão de tentar de novo, e os dados. E as duas ações de coleção: a estrela na barra superior (RF04) e o botão Capturar embaixo (RF07).
5:05	screens/ favorites_screen.dart screens/ captured_screen.dart	As duas telas são curtas de propósito: elas só fazem context.watch no provider. É por isso que, ao desfavoritar, a lista se atualiza sozinha, sem nenhum código de recarregar — que é exatamente o que o RF05 pede.
5:18	screens/ home_screen.dart	Por fim o RF08, no método _search: TextEditingController mais o botão Buscar, e ao encontrar ele já navega direto para a tela de detalhes, reaproveitando o detalhe que acabou de ser baixado em vez de pedir de novo à API.

5:30 – 8:00 — Alex Narok Stavasz: explicação técnica

Tema escolhido: Provider vs. setState — por que favoritos e capturados são estado global em vez de estado local de widget. Este é o tema que vai na Seção 4 do PDF de entrega, começando em **5:30**.

Tempo	Ponto a desenvolver
5:30	Enunciar o problema. A mesma marcação de “favorito” aparece em quatro lugares ao mesmo tempo: o selo no card do catálogo, a estrela na barra da tela de detalhes, a tela de Favoritos e a contagem no título. A pergunta é: quem é o dono dessa informação?
5:50	Por que setState não resolveria. Com setState, cada tela teria a sua própria cópia da lista. Ao favoritar na tela de detalhes, seria preciso devolver o resultado no Navigator .pop, a tela anterior teria que receber esse retorno e se reconstruir, e a tela de Favoritos — que está em outro ramo da pilha — nem ficaria sabendo. Cada nova tela multiplicaria esse trabalho.
6:20	Como resolvemos. O CollectionProvider é um ChangeNotifier registrado no MultiProvider em lib/main.dart. Existe uma única fonte da verdade e as telas apenas a observam. Mostrar o ChangeNotifierProxyProvider: ele amarra a coleção ao usuário logado, então trocar de conta troca a coleção automaticamente.
6:45	Detalhe fino: watch vs. select. As telas de lista usam context .watch, mas o card do catálogo usa context .select. A diferença importa: com watch, favoritar um Pokémon reconstruiria os 40 cards da grade; com select, só reconstrói o card cuja marcação mudou.
7:05	O que continua sendo setState — e por quê. Não globalizamos tudo. O texto digitado na busca e o _isSearching do HomeScreen, e o _future da tela de detalhes, continuam em setState: são estados que nascem e morrem com aquela tela e não interessam a mais ninguém. O critério que usamos foi esse: estado compartilhado entre telas vai para o Provider, estado de uma tela só fica local.
7:30	A consequência prática. Como o estado é único, a persistência também fica em um lugar só: o _apply grava no aparelho e sincroniza com a nuvem. Se o estado estivesse espalhado em setState, cada tela precisaria lembrar de salvar — e bastaria uma esquecer para os dados divergirem.
7:50	Encerramento. Repetir o link do repositório no GitHub, dizer que todos os dez requisitos funcionais foram implementados, mais os dois bônus (persistência em nuvem e autenticação real), e agradecer.

4. Se preferirem outro tema técnico

O enunciado permite escolher outro tema da lista. Estes três também têm base direta no código deste projeto — se trocarem, lembrem de atualizar a Seção 4 do PDF de entrega.

Tema	Onde o código sustenta o argumento
FutureBuilder vs. then/catchError	A tela de detalhes usa FutureBuilder porque é uma requisição com início e fim; o catálogo usa ChangeNotifier porque “Carregar mais” acumula resultados entre várias requisições, algo que um único Future não representa bem. Arquivos: detail_screen.dart e catalog_provider.dart.
Local vs. Nuvem	O merge em CollectionProvider ._load resolve conflito entre aparelho e nuvem pelo campo updated_at, e o SyncService só age quando há credenciais e usuário autenticado. Dá para falar de latência, funcionamento offline e das policies de RLS em supabase/schema.sql.
Tratamento de falhas de rede	O _getJSON traduz timeout, 404 e falha de socket em mensagens diferentes, e o _LoadMoreFooter mostra o erro sem descartar a lista já carregada. Dá para demonstrar ao vivo ligando o modo avião no emulador.

5. Checklist antes de gravar e antes de postar

	Item
☐	Repositório no GitHub criado e público — o link aparece no vídeo e no PDF.
☐	Emulador (ou celular) aberto, com internet, e o app deslogado , para a demonstração começar pela tela de login.
☐	Conta de teste funcionando e senha à mão, para não travar na hora de digitar.
☐	Gravar tela + câmera: os quatro integrantes precisam aparecer.
☐	Nomes completos ditos na abertura iguais aos da capa do PDF.
☐	Cada integrante citou pelo menos um arquivo que implementou.
☐	Duração final entre 6:00 e 8:00 (conferir antes de subir).
☐	Vídeo publicado no YouTube como Público e o link testado em uma aba anônima.
☐	Anotar no PDF de entrega: tema técnico escolhido e que ele começa em 5:30 .

Lembrete de penalidades (item 8). Entrega por comentário em vez do campo oficial: –30%. Cada item faltando no PDF (links, seções, tabela, screenshots): –20%. Atraso: –20% por semana. Nome de integrante fora da capa: zero para aquela pessoa. Vídeo fora da faixa de tempo: –5% por minuto.